Module 2 - Information Gathering and OSINT

2.1 - Recon Foudations

This section grounds you in the terminology, workflow, and Python primitives that underpin every subsequent OSINT or network-recon script. If you have already read *Black Hat Python 2nd Ed.*(chap. 2 + 3) the concepts will feel familiar—we re-implement them here with modern libraries (Python 3.12, `asyncio`, `httpx`, `aiodns`) and stricter OPSEC.

2.1.1 Active vs Passive Recon

Category	You touchthe target?	Typical Data Sources	OPSEC Impact
Passive	No	WHOIS, CRT.SH, Certificate Transparency Logs, Shodan*, public GitHub, leaked databases	Stealthy, no logs on target
Active	Yes	DNS queries, port scans, banner grabs, web crawling, SMTP VRFY	Generates network/HTTP logs on target, may trigger WAF/IPS

- Shodan itself collected data actively; **your query** is passive.

> **Rule #1:** Exhaust passive sources *first*—they cost nothing in exposure and often narrow active scope dramatically.

2.1.2 Canonical Recon Workflow

```
1. Scope Definition  ──► 2. Passive Recon ──► 3. Correlation & Pivot ──►
                                  │
```

```
                    ▼
                4. Active Recon
                    |
                    ▼
                5. Reporting / Import into Next Phase
```

- **Scope Definition** CIDR blocks, root domains, corporate aliases, employee lists.
- **Passive Recon** Collect all public artefacts.
- **Correlation & Pivot** Enrich, deduplicate, and generate new leads (e.g., ASN ⇒ new IP ranges).
- **Active Recon** Precise probing: live hosts, services, versions.
- **Reporting** Output clean JSON/CSV for exploitation modules or dashboards.

2.1.3 Core Python Libraries & Why They Matter

Library	Quick Purpose	Replacement for
asyncio / trio	Massive concurrency without threads	Twisted, raw threads
httpx (async)	HTTP/HTTPS requests with HTTP/2, proxy & SOCKS support	requests
aiodns	Fully-async DNS resolution (UDP + TCP + DNSSEC)	System resolver, dnspython sync
ipwhois	ASN, RIR, prefix look-ups	ARIN WHOIS manually
rich	Pretty terminal tables & spinners	Manual print debugging
pandas	Ad-hoc data wrangling & CSV export	Excel hell

Set up a dedicated virtualenv:

```
python3 -m venv recon
source recon/bin/activate
```

```
pip install httpx[http2] aiodns ipwhois rich pandas
```

2.1.4 Async Skeleton ("Black-Hat Recon 2025")

Below is a distilled *Black Hat Python* style skeleton—but using `asyncio` and type hints:

```
#!/usr/bin/env python3
"""
bhp_recon_async.py — Passive-first recon scaffold
"""

from __future__ import annotations
import asyncio, socket, ssl, json, ipaddress
from pathlib import Path
import httpx, aiodns
from rich.progress import Progress

RESOLVER = aiodns.DNSResolver()
HTTP     = httpx.AsyncClient(http2=True, timeout=10)

# ──────────────────────────────────────────────────────────────
async def resolve(host: str) -> list[str]:
    """Return A/AAAA records or [] on failure."""
    try:
        ans = await RESOLVER.query(host, "A")
        return [r.host for r in ans]
    except aiodns.error.DNSError:
        return []

async def grab_banner(ip: str, port: int) -> str:
    """Very naive TCP banner."""
```

```python
    try:
        reader, writer = await asyncio.open_connection(ip, port)
        writer.write(b"\r\n")
        await writer.drain()
        data = await asyncio.wait_for(reader.read(128), 4)
        writer.close(); await writer.wait_closed()
        return data.decode(errors="replace").strip()
    except Exception:
        return ""

async def probe_host(host: str, ports: list[int]) -> dict:
    ips   = await resolve(host)
    tasks = [grab_banner(ip, p) for ip in ips for p in ports]
    banners = await asyncio.gather(*tasks)
    return {"host": host, "ips": ips, "banners": banners}

# ──────────────────────────────────────────────────────────────
async def main(wordlist: Path, root_domain: str) -> None:
    ports   = [80, 443, 22, 21]
    subs    = (root_domain if line.startswith("#") else f"{line.strip()}.{root_domain}"
               for line in wordlist.read_text().splitlines())
    async with Progress() as bar:
        tasks = [probe_host(sub, ports) for sub in subs]
        results = []
        for coro in asyncio.as_completed(tasks):
            res = await coro
            results.append(res)
            bar.update(bar.add_task(res["host"]), advance=1)
    Path("recon.json").write_text(json.dumps(results, indent=2))
    print("[+] Saved recon.json")

if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser()
```

```
    ap.add_argument("domain"), ap.add_argument("wordlist", type=Path)
    asyncio.run(main(ap.parse_args().wordlist, ap.parse_args().domain))
```

Highlights

- **Concurrency**— `asyncio.gather` launches hundreds of DNS/Banner tasks without threads.
- **Graceful failure**— `except Exception` inside helpers prevents one failed banner from killing the run.
- **Output**—structured JSON ready for enrichment in 2.3–2.10.

2.1.5 Correlation Techniques

After collecting raw hosts/ports, enhance:

Enhancement	Quick Python Tactic
Reverse DNS	socket.gethostbyaddr(ip) asynchronously via thread pool
ASN	ipwhois.IPWhois(ip).lookup_rdap()
Service Detect	Parse banner, use httpx for Server header, TLS ALPN
Screenshot	Call chromium --headless via subprocess or use selenium-wire

Store enriched rows in a `pandas.DataFrame` → export to CSV/HTML.

2.1.6 Rate Limiting & Anti-Fingerprinting

- Limit outbound requests with `asyncio.Semaphore(10)`; sleep jitter 50–200 ms.
- Rotate source IPs (Tor/Proxy) if scanning hostile territory.

- Randomise TLS JA3 (use `tls-client` wrapper) to avoid WAF fingerprints.

2.1.7 Exercises

1. **Passive-Only Recon** Write `crtsh_subs.py` that queries `https://crt.sh/?q=%.example.com&output=json` and dumps unique sub-domains.
2. **Concurrent Banner Grab** Extend the skeleton to add TLS SNI support (`ssl.create_default_context()`) and extract certificate CN/SANs.
3. **Pivot Script** Given a list of IPs, auto-expand to ASN prefixes and push back through the passive recon pipeline.
4. **OPSEC Check** Instrument your script with `rich` to log request/second; tune concurrency until you get < 1 RPS at network edge.
5. **Unit Tests** Use `pytest-asyncio` to mock DNS and TCP services, ensuring `probe_host()` handles NXDOMAIN and timeout correctly.

2.2 - DNS & Sub-Domain Enumeration

This section dives into DNS mechanics and shows how to discover an organisation's sub-domains with **Python-only tooling**. The material extends and modernises the "DNS Recon" scripts from *Black Hat Python 2nd Ed.*, adding asyncio concurrency, DNSSEC awareness, and Certificate-Transparency (CT) pivots.

2.2.1 DNS Recap

Record	Purpose	Recon Value
A / AAAA	Host → IPv4 / IPv6	Maps names to addresses for later scans
CNAME	Alias to another name	Chains expose CDNs or legacy hosts
NS	Authoritative servers	May hint at internal naming schemes
MX	Mail exchange	Outsourced mail (G Suite / O-365)
TXT	SPF / DMARC / misc	Cloud providers, validation tokens
SRV	Service location	SIP, LDAP, Kerberos endpoints

Record	Purpose	Recon Value
DNSSEC	Auth signatures	Mis-configs enable downgrade attacks

2.2.2 Passive Enumeration Workflows

1. **CT Logs** – `crt.sh`, `tls.bufferover.run`.
2. **Historical DNS** – DNSDB, SecurityTrails.
3. **Search-Engine Dorks** – `site:*.example.com -www`.

2.2.3 Active Enumeration Techniques

Technique	Python Tooling	Caution
Wordlist brute-force	asyncio + aiodns	Detect wildcards first
Zone transfer (AXFR)	dnspython.xfr()	Loud; rarely allowed
NSEC walk	dnspython	Works only on NSEC-enabled zones
SRV sweep	Query _service._tcp patterns	Finds hidden infra
Google DNS JSON	https://dns.google/resolve	Useful behind firewalls

---

2.2.4 Detecting Wildcard DNS

```python
import random, string, asyncio, socket, aiodns
resolver = aiodns.DNSResolver()

async def is_wildcard(root: str) -> tuple[bool, set[str]]:
    """
    Return (True, {ips}) if root domain uses wildcard A records.

    Generates a random, guaranteed-nonexistent sub-domain and
    attempts to resolve it.
    """
    # create a 20-char random label unlikely to exist
    token = ''.join(random.choices(string.ascii_lowercase, k=20))
    try:
        # query A record; AF_INET = IPv4
        ans = await resolver.gethostbyname(f"{token}.{root}", socket.AF_INET)
        ips = {r.host for r in ans}
        return True, ips
    except aiodns.error.DNSError:
        # NXDOMAIN → no wildcard
        return False, set()
```

2.2.5 Async Brute-Forcer (modern rewrite of *Black Hat Python* example)

```python
#!/usr/bin/env python3
"""
sub_bruter.py  —  Asynchronous DNS brute-forcer with wildcard filtering.

Usage:
  python sub_bruter.py example.com subdomains.txt -c 800
"""
```

```python
import asyncio, socket, argparse, sys
from pathlib import Path
import aiodns

resolver = aiodns.DNSResolver()

# ──────────────────────────────────────────────────────────────
async def resolve(name: str) -> list[str]:
    """Return list of IPv4 addresses for *name* or empty list if NXDOMAIN."""
    try:
        # Query A records; other types can be added if needed
        response = await resolver.gethostbyname(name, socket.AF_INET)
        return [r.host for r in response]
    except aiodns.error.DNSError:
        return []

# ──────────────────────────────────────────────────────────────
async def worker(q: asyncio.Queue, root: str, wildcard_ips: set[str]) -> None:
    """
    Consumer coroutine:
      1. Pull sub-domain from queue
      2. Resolve FQDN
      3. Print only non-wildcard results
    """
    while True:
        sub = await q.get()
        fqdn = f"{sub}.{root}"
        ips  = await resolve(fqdn)

        # Print when resolution succeeded *and* IPs differ from wildcard set
        if ips and set(ips) != wildcard_ips:
            print(f"{fqdn:<40} {', '.join(ips)}")
```

```python
        q.task_done()

# ────────────────────────────────────────────────────────────────────────
async def main(wordlist: Path, root: str, concurrency: int) -> None:
    """Set up the queue, spawn workers and wait for completion."""
    # Test for wildcard DNS once and reuse result
    wildcard_present, wildcard_ips = await is_wildcard(root)
    if wildcard_present:
        print(f"[!] Wildcard detected → {', '.join(wildcard_ips)}")

    queue = asyncio.Queue()

    # Pre-load queue with every sub-label from wordlist
    for sub in wordlist.read_text().splitlines():
        await queue.put(sub.strip())

    # Spawn worker coroutines
    tasks = [asyncio.create_task(worker(queue, root, wildcard_ips))
             for _ in range(concurrency)]

    # Wait until queue fully processed
    await queue.join()

    # Gracefully cancel workers (otherwise they await forever)
    for t in tasks:
        t.cancel()

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("domain")
    ap.add_argument("wordlist", type=Path)
    ap.add_argument("-c", "--concurrency", type=int, default=500,
                    help="number of simultaneous DNS queries")
    args = ap.parse_args()
```

```
    try:
        asyncio.run(main(args.wordlist, args.domain, args.concurrency))
    except KeyboardInterrupt:
        sys.exit(1)
```

Key points

- **Concurrency** via `asyncio.create_task` – thousands of queries/second without threads.
- **Wildcard filter** – drop false positives when `*.example.com` is configured.
- **Cancellation** – tasks are cancelled after queue drains to avoid pending awaits.

2.2.6 Zone-Transfer Helper (AXFR)

```
import dns.query, dns.zone

def attempt_axfr(domain: str, ns: str) -> list[str]:
    """
    Try a full zone transfer (AXFR) against *ns*.
    Returns list of hostnames if successful; else empty list.
    """
    try:
        # Request AXFR; may require TCP fallback (handled internally)
        zone = dns.zone.from_xfr(dns.query.xfr(ns, domain, timeout=5))
        return [str(node) for node in zone.nodes]
    except Exception:
        # Any failure (REFUSED, timeout, etc.) → treat as closed zone
        return []
```

2.2.7 CT-Log Harvester (crt.sh)

```python
import httpx, asyncio, re

async def fetch_ct(domain: str) -> set[str]:
    """
    Query crt.sh JSON endpoint and return unique sub-domains
    stripped of leading wildcard.
    """
    url = f"https://crt.sh/?q=%25.{domain}&output=json"

    async with httpx.AsyncClient(timeout=30) as client:
        r = await client.get(url)
        r.raise_for_status()               # bubble up HTTP errors

    # Each JSON object contains 'name_value'; may include wildcards & duplicates
    raw_names = {entry["name_value"].lower() for entry in r.json()}

    # Remove leading '*.' and keep only entries ending with target domain
    cleaned = {re.sub(r'^\*\.', '', n) for n in raw_names if n.endswith(domain)}
    return cleaned
```

2.2.8 SRV Sweep Example

```python
common_srv = [
    ("_sip", "_tcp"),
    ("_sip", "_udp"),
    ("_ldap", "_tcp"),
    ("_autodiscover", "_tcp"),
]
```

```python
async def srv_scan(domain: str) -> dict[str, list[str]]:
    """
    Query common SRV records and return mapping:
      '_service._proto' → [targets]
    """
    results = {}
    for service, proto in common_srv:
        fqdn = f"{service}.{proto}.{domain}"
        try:
            ans = await resolver.query(fqdn, "SRV")
            targets = [f"{r.target.decode()[:-1]}:{r.port}" for r in ans]
            results[f"{service}.{proto}"] = targets
        except aiodns.error.DNSError:
            pass
    return results
```

2.2.9 Organising and Merging Results

```python
import pandas as pd, json

# Read CSV from brute-force and JSON from CT logs
brute_df = pd.read_csv("brute.csv")
ct_hosts = json.loads(Path("crtsh.json").read_text())

# Build DF from CT set and concat
ct_df = pd.DataFrame({"host": list(ct_hosts), "source": "crtsh"})
merged = pd.concat([brute_df, ct_df], ignore_index=True)

# Drop duplicates, keeping first occurrence
```

```
merged = merged.drop_duplicates(subset="host")
merged.to_csv("merged.csv", index=False)
```

2.2.10 OPSEC & Rate-Limiting Tips

- **Semaphore throttle**

```
sem = asyncio.Semaphore(100)   # max 100 queries in flight
async with sem:
    await resolver.query(...)
```

- Introduce random sleep between bursts to mimic user traffic.
- Consider Tor (`socks5h://127.0.0.1:9050`) for DNS tunnels in hostile tests.

2.2.11 Hands-On Exercises

#	Challenge	Hint
1	Extend sub_bruter.py to resolve AAAA (IPv6) and merge with A results.	socket.AF_INET6
2	Write zonehunter.py that loops through each NS returned by dig NS and calls attempt_axfr.	dns.resolver.resolve()
3	Build a NetworkX graph linking hosts that share an IP. Export GraphML.	nx.Graph(); g.add_edge(host, ip)
4	Enhance wildcard detection to support multiple wildcard IPs (some CDNs return rotators).	Compare set(ips) difference
5	Use pytest-asyncio to mock httpx calls to crt.sh and ensure duplicate name_value entries are deduped.	respx library

2.3 - WHOIS, ASN & Geo-IP Enrichment

Convert raw IPs from the previous sub-domain phase into rich, pivot-ready intelligence. All snippets are heavily commented so you can paste-and-run.

2.3.0 Why This Matters

- **Attribution** – Know which cloud, hosting or ISP owns a box before you attack.
- **Pivoting** – An ASN or organisation name often reveals *additional* CIDR blocks to enumerate.
- **Reporting** – Stakeholders love maps and tables ("78 % of assets sit in AWS us-east-1").

We will rely on:

Task	Library	Reason
RDAP / Legacy WHOIS	ipwhois	Pure-Python, no shell calls
Geo-IP	geoip2 (GeoLite2)	Free, local DB → no external traffic
BGP Prefix Expansion	ipwhois, bgpq4* (optional)	Builds larger scopes

* *If `bgpq4` is not available on your OS, skip the prefix-expansion extra credit.*

2.3.1 RDAP vs Port 43 "Legacy" WHOIS

- **RDAP** (RESTful) returns structured JSON (owner, contacts, routes).
- Port 43 plaintext WHOIS is still useful when RDAP is rate-limited, but parsing is messy.
- `ipwhois` automatically tries RDAP first, then falls back.

2.3.2 Single-Look-Up Example (synchronous)

```python
from ipwhois import IPWhois

ip = "104.26.10.78"                        # cdn.cloudflare.net
obj = IPWhois(ip)
result = obj.lookup_rdap(depth=1)          # depth=1 → do not recurse into parent nets

# Pretty-print key bits
print(result["network"]["cidr"])           # '104.16.0.0/13'
print(result["asn_description"])           # 'CLOUDFLARENET'
print(result["asn_country_code"])          # 'US'
```

2.3.3 Bulk Async RDAP (speed-up for hundreds/thousands of IPs)

`ipwhois` is blocking; wrap in a thread pool so the event loop stays responsive.

```python
#!/usr/bin/env python3
"""
bulk_rdap.py  —  Resolve a list of IPs to ASN, Org, Country concurrently.

    python bulk_rdap.py ips.txt rdap.csv --workers 50
"""
from __future__ import annotations
import csv, sys, argparse, concurrent.futures
from pathlib import Path
from ipwhois import IPWhois, exceptions as ipw_exc
from rich.progress import Progress, BarColumn, TimeElapsedColumn
```

```python
def rdap_lookup(ip: str) -> dict[str, str] | None:
    """
    Look up *ip* via RDAP; return dict with minimal fields or None on error.
    Called inside a worker thread → keep it CPU/blocking friendly.
    """
    try:
        data = IPWhois(ip).lookup_rdap(depth=1)
        return {
            "ip": ip,
            "asn": data["asn"],
            "asn_desc": data["asn_description"],
            "country": data["asn_country_code"],
            "cidr": data["network"]["cidr"],
        }
    except (ipw_exc.IPDefinedError, ipw_exc.HTTPLookupError):
        # RFC1918 or RDAP rate-limit — skip
        return None

def main(inp: Path, out: Path, workers: int = 30) -> None:
    ips = [line.strip() for line in inp.read_text().splitlines() if line.strip()]
    with Progress("[progress.description]{task.description}",
                  BarColumn(), "[magenta]{task.completed}/{task.total}",
                  TimeElapsedColumn()) as bar:
        task = bar.add_task("[cyan]RDAP", total=len(ips))
        results: list[dict[str, str]] = []

        with concurrent.futures.ThreadPoolExecutor(max_workers=workers) as pool:
            for res in pool.map(rdap_lookup, ips):
                bar.advance(task)
                if res:
                    results.append(res)

    # Write CSV
    with out.open("w", newline="") as fh:
```

```
        writer = csv.DictWriter(fh, fieldnames=results[0].keys())
        writer.writeheader()
        writer.writerows(results)

    print(f"[+] Saved {len(results)} enriched rows → {out}")

if __name__ == "__main__":
    ap = argparse.ArgumentParser()
    ap.add_argument("infile", type=Path)
    ap.add_argument("outfile", type=Path)
    ap.add_argument("--workers", "-w", type=int, default=30)
    args = ap.parse_args()
    try:
        main(args.infile, args.outfile, args.workers)
    except KeyboardInterrupt:
        sys.exit(1)
```

What the comments cover

- Thread pool because RDAP performs network I/O **and** JSON parsing.
- Gracefully skips RFC 1918 or rate-limited responses, so the script *never crashes*.
- Progress bar (`rich`) for user feedback.

2.3.4 Local Geo-IP (no outbound traffic)

1. Grab the **GeoLite2-Country** database (free, requires MaxMind account).
2. Install `geoip2` :

```
pip install geoip2
```

1. Integration snippet:

```
import geoip2.database
reader = geoip2.database.Reader("GeoLite2-Country.mmdb")

def country_iso(ip: str) -> str | None:
    """
    Return ISO code ('US', 'BR', …) or None if not in DB.
    """
    try:
        return reader.country(ip).country.iso_code
    except geoip2.errors.AddressNotFoundError:
        return None
```

> **Tip**: keep one global `Reader` instance; opening the MMDB repeatedly is slow.

2.3.5 ASN → Prefix Expansion *(optional advanced pivot)*

Once you know an ASN, you can programmatically fetch all its advertised prefixes:

```
from ipwhois.utils import get_asn_radix

def prefixes_for_asn(asn: str) -> list[str]:
    """
    Return list like ['1.1.1.0/24', '1.0.0.0/24'] for AS13335.
    """
    try:
        # Queries Team Cymru radix server (port 43); fast plaintext
        return get_asn_radix(asn)
```

```
    except Exception:
        return []
```

Merge the list of new prefixes into your scope definition—then feed back into the *mass scan* pipeline.

2.3.6 Putting It Together – Enrichment Pipeline

```
raw_ips.txt                ← Output from Module 2.2 port/banners
      │
      ├─► bulk_rdap.py      → rdap.csv      (ASN, CIDR, Country)
      │
      ├─► geoip_enrich.py   → geo.csv
      │
      └─► pandas merge      → ip_enriched.csv
```

Example merge:

```
import pandas as pd
rdap = pd.read_csv("rdap.csv")
geo  = pd.read_csv("geo.csv")
merged = pd.merge(rdap, geo, on="ip", how="left")
merged.to_csv("ip_enriched.csv", index=False)
```

2.3.7 Quick Visual – Top 10 ASNs

(No plotting code shown here; later you will use matplotlib in a user-visible notebook.)

```
top_asn = merged["asn_desc"].value_counts().head(10)
print(top_asn)
```

2.3.8 Handling Rate Limits & Caching

- **RDAP** servers throttle hard (~100 req/min).
- Cache successful look-ups in `sqlite3`:

```
import sqlite3, json, time
db = sqlite3.connect("rdap_cache.db")
db.execute("CREATE TABLE IF NOT EXISTS rdap (ip TEXT PRIMARY KEY, ts INTEGER, data TEXT)")
```

- **Strategy**: read from cache first; if miss or older than 7 days, refetch.

2.3.9 Exercises

#	Task	Hint
1	Enhance bulk_rdap.py to retry once after HTTPLookupError with exponential back-off.	time.sleep(2 ** attempt)
2	Add GeoLite2 enrichment in the same thread pool so each worker returns both ASN and country.	Return a merged dict
3	Write asn_pivot.py that reads ip_enriched.csv, groups by ASN, pulls extra prefixes, writes new_scope.txt.	Deduplicate CIDRs
4	Build a bar chart “Assets per Country” using matplotlib.	DataFrame .value_counts()

#	Task	Hint
5	Unit-test rdap_lookup() with pytest and requests-mock to simulate a 429 → retry flow.	Patch ipwhois.net.get_http_json()

2.3.10 Ready for 2.4?

You should now be able to:

- Convert an IP to **CIDR, org, ASN, country** locally and at scale.
- Cache and throttle look-ups to avoid RDAP bans.
- Feed ASN prefixes back into recon scope.
- Produce CSV ready for dashboards or mapping tools.

2.4 - Banner Grabbing & Service Fingerprinting

Turn each "IP

" from earlier modules into real, actionable service data. All code is Python 3.12+, fully commented, and designed to run without third-party binaries.

2.4.0 Why banners?

Goal	What the banner gives you
Vulnerability mapping	Exact software name & version (OpenSSH_7.4p1)
Privilege escalation	Misconfig clues (vsFTPd 2.3.4)

Goal	What the banner gives you
False-positive pruning	Confirm port 80 is truly HTTP, not a honeypot
Social engineering	Company mail server string reveals vendor

2.4.1 Strategy & Library Stack

Layer	Library	Reason
Raw TCP / TLS	asyncio + ssl	No dependencies, 10k+ parallel sockets
HTTP/HTTPS	httpx (async)	Auto redirects, HTTP/2, proxies
SSH / SMTP hints	Plain socket with protocol “canned strings”	Lightweight
JA3 TLS hash	ja3-python or fallback manual MD5 of ClientHello	Fingerprint WAF/CDN

Install:

```
pip install httpx[http2] ja3
```

2.4.2 Generic Async “Grab” Function

```
# grab_core.py — reusable primitives
import asyncio, ssl, socket, hashlib, struct
from contextlib import suppress
from typing import NamedTuple
```

```python
class Banner(NamedTuple):
    ip: str
    port: int
    proto: str      # e.g. "http", "ssh", "tls"
    banner: str     # printable banner / header line
    extra: dict     # anything else (JA3, cert CN…)

# ────────────────────────────────────────────────────────────────
async def tcp_banner(ip: str, port: int, payload: bytes = b"\r\n",
                     read_size: int = 256, timeout: float = 4.0) -> bytes | None:
    """
    Open a naked TCP socket, optionally send *payload*, read up to *read_size*.
    Returns raw bytes or None on timeout/error.
    """
    try:
        reader, writer = await asyncio.wait_for(
            asyncio.open_connection(ip, port), timeout)
        # Send something minimal to provoke banner (SMTP, FTP need newline)
        writer.write(payload)
        await writer.drain()
        data = await asyncio.wait_for(reader.read(read_size), timeout)
        writer.close(); await writer.wait_closed()
        return data
    except Exception:
        return None

# ────────────────────────────────────────────────────────────────
async def tls_hello(ip: str, port: int, timeout: float = 5.0) -> tuple[str, str] | None:
    """
    Perform TLS handshake, return (JA3 MD5, certificate CN) or None.
    """
    context = ssl.create_default_context()
    context.set_ciphers("ALL")
    # Fingerprint TLS ClientHello → ja3 MD5
```

```
    try:
        reader, writer = await asyncio.wait_for(
            asyncio.open_connection(ip, port, ssl=context,
                                    server_hostname=ip), timeout)
        # ja3 lib works on ssl.SSLSocket; _sslproto is private → simpler manual hash
        # python-ja3 fallback:
        with suppress(ImportError):
            from ja3 import TLSFingerprint
            fp = TLSFingerprint.from_socket(writer.get_extra_info("ssl_object"))
            ja3 = fp.ja3_hash
        # get certificate CN
        cert = writer.get_extra_info("ssl_object").getpeercert()
        cn = None
        if cert:
            subject = dict(x[0] for x in cert["subject"])
            cn = subject.get("commonName")
        writer.close(); await writer.wait_closed()
        return ja3, cn
    except Exception:
        return None
```

Comments explain each step, timeout handling, and graceful error swallowing so one bad host never kills the coroutine swarm.

2.4.3 HTTP/HTTPS Scraper

```
import httpx, asyncio, re

async def http_head(ip: str, port: int, tls: bool = False) -> Banner | None:
    """
    Send a minimal HEAD request, return Banner tuple with
    Server header and optional title tag.
```

```
    """
    url = f"http{'s' if tls else ''}://{ip}:{port}/"
    try:
        async with httpx.AsyncClient(
            http2=True, verify=False, timeout=5.0, follow_redirects=True
        ) as cli:
            r = await cli.head(url)
        server = r.headers.get("server", "").strip()
        # Attempt GET only if HEAD lacks info
        title = ""
        if not server:
            r2 = await cli.get(url, follow_redirects=True)
            m = re.search(r"<title.*?>(.*?)</title>", r2.text, re.I | re.S)
            if m:
                title = m.group(1).strip()
        banner = server or title or f"{r.status_code} {r.reason_phrase}"
        return Banner(ip, port, "https" if tls else "http", banner, {})
    except Exception:
        return None
```

Why HEAD? Quick, often enough; fallback GET only when needed.

2.4.4 Putting It Together – Concurrent Scanner

```
#!/usr/bin/env python3
"""
scanner.py  —  Enumerate banners for IP:port lines.

input.txt  -> 192.0.2.1:22
              203.0.113.10:443
"""
```

```python
import asyncio, json, random
from pathlib import Path
from grab_core import tcp_banner, tls_hello, http_head, Banner

PORT_HINTS = {
    22:  ("ssh",  b"\r\n"),
    25:  ("smtp", b"EHLO example.com\r\n"),
    21:  ("ftp",  b"\r\n"),
    80:  ("http", b"HEAD / HTTP/1.0\r\n\r\n"),
    443: ("https", b"HEAD / HTTP/1.0\r\n\r\n"),
}

async def fingerprint(ip: str, port: int) -> Banner | None:
    """Dispatch correct grab method based on port hint table."""
    if port in (80,):
        return await http_head(ip, port, tls=False)
    if port in (443,):
        # Try HTTPS; also capture JA3 + CN
        http_banner = await http_head(ip, port, tls=True)
        xtra = await tls_hello(ip, port)
        if http_banner and xtra:
            http_banner.extra["ja3"], http_banner.extra["cn"] = xtra
        return http_banner
    # Fallback raw TCP
    proto, pay = PORT_HINTS.get(port, ("tcp", b"\r\n"))
    data = await tcp_banner(ip, port, pay)
    if data:
        try:
            banner = data.decode(errors="replace").strip()
        except UnicodeDecodeError:
            banner = repr(data[:30])
        return Banner(ip, port, proto, banner, {})
    return None
```

```python
async def run(file: Path, out: Path, concurrency: int = 500):
    lines = [l.strip() for l in file.read_text().splitlines() if l.strip()]
    q     = asyncio.Queue()
    for line in lines:
        ip, p = line.split(":")
        await q.put((ip, int(p)))

    results = []

    async def worker():
        while True:
            ip, port = await q.get()
            res = await fingerprint(ip, port)
            if res:
                print(f"{res.ip}:{res.port:<5} {res.proto:<5} {res.banner[:60]}")
                results.append(res._asdict())
            # jitter to evade IDS thresholds
            await asyncio.sleep(random.uniform(0.02, 0.08))
            q.task_done()

    tasks = [asyncio.create_task(worker()) for _ in range(concurrency)]
    await q.join()
    for t in tasks: t.cancel()

    out.write_text(json.dumps(results, indent=2))
    print(f"[+] Saved {len(results)} banners → {out}")

if __name__ == "__main__":
    import argparse, sys
    ap = argparse.ArgumentParser()
    ap.add_argument("infile", type=Path)
    ap.add_argument("outfile", type=Path)
    ap.add_argument("-c", "--concurrency", type=int, default=400)
    args = ap.parse_args()
```

```
    try:
        asyncio.run(run(args.infile, args.outfile, args.concurrency))
    except KeyboardInterrupt:
        sys.exit(0)
```

Commentary highlights

- **Port hint table** – lets you add new protocols easily (`3389` , `6379` , `27017` etc.).
- **Random jitter** between requests defeats simple RPS detectors.
- Saves **JSON** with everything serialisable (`_asdict()` on `NamedTuple`).

2.4.5 Interpreting TLS JA3 & Cert CN

- **JA3 hash** — MD5 of ClientHello parameters; compare against known WAF/CDN fingerprints.
- **CN / SAN** — Confirms virtual-host name (`*.corp.example.com`) even when DNS is hidden.

Store these in `extra` field; later modules will correlate JA3 → vendor mapping.

2.4.6 Output Example

```
{
  "ip": "192.0.2.13",
  "port": 443,
  "proto": "https",
  "banner": "cloudflare",
  "extra": {
    "ja3": "5d1c6b...",
    "cn": "ssl123456.cloudflare.com"
```

```
  }
}
```

2.4.7 Rate-Limit & OPSEC Advice

- **Throttle** to < 100 new TCP connections per second per destination ASN.
- Use **SOCKS5 or TOR** proxies if testing hostile network.
- Disable SNI for ambiguous hosts to avoid HTTP 302 redirect honeypots.
- Never send fingerprints containing org-specific strings (`EHLO target.com`) in red-team ops.

2.4.8 Exercises

#	Challenge	Hint
1	Add Redis (6379) support: send b"INFO\r\n" and parse redis_version: line into banner.	Use regex
2	Extend TLS handler to calculate **JA3S** (server fingerprint) by hashing ServerHello.	ja3 library has .ja3s_hash
3	Write a pandas script that merges ip_enriched.csv (Module 2.3) with banner JSON, producing “Org → Service list”.	pd.merge on ip
4	Build a report that flags any service version older than a supplied CVE list (e.g., OpenSSH < 8.2).	Semantic-version compare
5	Unit-test tcp_banner() using pytest + asyncio.start_server to serve known banner and assert output.	pytest-asyncio

2.5 - Shodan and Censys

“Query the Internet’s telescopes instead of rescanning the whole sky.”

This module shows you how to pull **machine-readable** data from the three most popular Internet-wide scanners, merge it with the IP lists you already have, and respect API quotas so you *never* burn a key in the middle of an engagement.

2.5.0 Why external scanners?

Benefit	Explanation
Speed	Tens of millions of hosts per minute (somebody else already did the probe)
Stealth / OPSEC	No packets from *your* IP — only harmless API calls
Historical context	See ports that were open weeks ago but are firewalled today
Rich metadata	Full TLS certs, banners, geolocation, reverse DNS

2.5.1 Environment Setup

```
pip install \
    shodan \
    censys==2.* \
    zoomeye \
    rich \
    tenacity     # smart exponential-back-off
```

> **Never** hard-code secrets in source.
> Export them once, your shell remembers:

```
export SHODAN_KEY="SHODAN-XXXXXXXXXXXXXXXXXXXXXXXX"
export CENSYS_ID="aaaaaaaa"
export CENSYS_SECRET="bbbbbbbbbbbbbbbbbbbbbbbbbbbbbb"
export ZOOMEYE_KEY="cccccccccccccccccccccccccccccccc"
```

2.5.2 Design goals for all pull-scripts

1. **Idempotent:** Running twice appends or updates safely (no duplicate JSON blobs).
2. **Retry-safe:** Transient `429`, `502`, `504` are handled automatically.
3. **Streamed writing:** Results land on disk *while* the crawl runs, so a crash costs at most one page.
4. **Provider-agnostic:** We normalise Shodan, Censys, ZoomEye into the *same* minimal schema:

```
{ip, port, transport, timestamp, org, banner, source}
```

2.5.3 Shodan Pull – fully commented

```
#!/usr/bin/env python3
"""
shodan_pull.py
~~~~~~~~~~~~~~
Streams Shodan results into NDJSON (one JSON doc per line).

Why NDJSON?
• Append-friendly ( `>>` concatenate files, no commas to fix )
• Tools like jq / Elasticsearch / Splunk ingest line-by-line.
"""
```

```python
from __future__ import annotations

import json
import os
import sys
import time
from pathlib import Path
from typing import Iterable, Iterator, List

import shodan                      # official SDK
from rich.progress import Progress # pretty progress bar
from tenacity import (             # robust retries
    retry,
    stop_after_attempt,
    wait_exponential,
)

SHODAN_KEY = os.getenv("SHODAN_KEY")
if not SHODAN_KEY:
    sys.exit("[-] SHODAN_KEY env var not set")

api = shodan.Shodan(SHODAN_KEY)


# ------------------------------------------------------------------------ #
#  Helper — one resilient query                                            #
# ------------------------------------------------------------------------ #
@retry(
    wait=wait_exponential(multiplier=2, min=2, max=30),  # 2s, 4s, 8s, …
    stop=stop_after_attempt(4),                          # give up after 4 tries
    reraise=True,
)
def run_query(q: str) -> Iterator[dict]:
```

```python
    """
    Yield banner dictionaries for *q* indefinitely.

    • Uses search_cursor() to bypass the 10k-result soft cap.
    • `retry` decorator makes 502/504 entirely transparent to caller.
    """
    for banner in api.search_cursor(q):
        yield banner


# ---------------------------------------------------------------------------- #
#  Transform Shodan JSON → universal schema                                    #
# ---------------------------------------------------------------------------- #
def normalise(b: dict) -> dict:
    """
    Map Shodan field names to our common schema.

    Shodan example keys used:
      • ip_str, port, transport, timestamp, org, data
    """
    return {
        "ip":         b.get("ip_str"),
        "port":       b.get("port"),
        "transport":  b.get("transport", "tcp"),
        "timestamp":  b.get("timestamp"),
        "org":        b.get("org") or b.get("asn"),
        "banner":     (b.get("data") or "").strip()[:8000],  # truncate garbage
        "source":     "shodan",
    }


# ---------------------------------------------------------------------------- #
#  Main driver                                                                 #
# ---------------------------------------------------------------------------- #
```

```python
def shodan_pull(queries: list[str], outfile: Path) -> None:
    tmp = outfile.with_suffix(".tmp")

    with tmp.open("w") as fh, Progress() as bar:
        task = bar.add_task("[cyan]Shodan", total=len(queries))
        for q in queries:
            try:
                for banner in run_query(q):
                    fh.write(json.dumps(normalise(banner)) + "\n")
            except Exception as e:
                bar.console.print(f"[red]Query failed:[/] {q} — {e!s}")
            bar.advance(task)

    tmp.replace(outfile)  # atomic rename → final file
    print(f"[+] Shodan results saved to {outfile} ({outfile.stat().st_size/1e6:.1f} MB)")


if __name__ == "__main__":
    shodan_pull(
        queries=[
            # Example queries — tweak as needed
            "product:OpenSSH port:22",
            'ssl:"CN=*.corp.example.com" port:443',
        ],
        outfile=Path("shodan.ndjson"),
    )
```

Key inline comments explained

- `tenacity.retry` handles network flakiness; exponential back-off prevents *hammering* Shodan and exhausting credits.
- `normalise()` converts provider-specific keys into the same output schema you'll reuse for Censys & ZoomEye.
- **Atomic** `replace` ensures you never leave a half-written file if the script is killed during rename.

2.5.4 Censys v2 Pull – fully commented

```python
#!/usr/bin/env python3
"""
censys_pull.py
~~~~~~~~~~~~~~
Queries Censys Search v2 (Hosts index) and writes CSV.

Pagination: 100 results per API call (max allowed).
"""

from __future__ import annotations
import csv, os, sys
from pathlib import Path
from typing import Iterator

from censys.search import CensysHosts
from rich.console import Console
from rich.progress import track
from tenacity import retry, stop_after_attempt, wait_exponential

CID = os.getenv("CENSYS_ID")
CSECRET = os.getenv("CENSYS_SECRET")
if not (CID and CSECRET):
    sys.exit("[-] Set CENSYS_ID and CENSYS_SECRET env vars")

hosts = CensysHosts(api_id=CID, api_secret=CSECRET)
console = Console()

# Normalisation identical to Shodan's
def norm(hit: dict) -> dict:
    """
```

```python
    Flatten Censys 'services' list (we record one line per service).
    """
    base = {
        "ip":        hit["ip"],
        "timestamp": hit["updated_at"],
        "org":       hit.get("autonomous_system", {}).get("name"),
        "source":    "censys",
    }
    for svc in hit["services"]:
        yield {
            **base,
            "port":      svc["port"],
            "transport": svc["transport_protocol"],
            "banner":    (svc.get("banner") or "").strip()[:8000],
        }

@retry(wait=wait_exponential(1, 2, 30), stop=stop_after_attempt(4), reraise=True)
def pages(q: str, per_page: int = 100) -> Iterator[dict]:
    """Lazy generator over all pages."""
    for page in hosts.search(q, per_page=per_page):
        yield from page

def run(q: str, out: Path) -> None:
    fieldnames = ["ip", "port", "transport", "timestamp", "org", "banner", "source"]
    with out.open("w", newline="") as csvfile:
        writer = csv.DictWriter(csvfile, fieldnames)
        writer.writeheader()

        total = hosts.count(q)
        for hit in track(pages(q), total=total, description=f"[cyan]{q}"):
            for row in norm(hit):
                writer.writerow(row)

    console.print(f"[green]+[/] Censys CSV → {out}")
```

```
if __name__ == "__main__":
    run(
        q='services.service_name: "HTTP" AND location.country: "BR"',
        out=Path("censys.csv"),
    )
```

Important comments show how we:

- **Flatten** Censys' nested `services` array into multiple CSV rows.
- Use `hosts.count()` to set progress-bar length ahead of time.
- Retry only *page* fetches, not individual row normalisation.

2.5.5 ZoomEye Pull – fully commented (async)

```
#!/usr/bin/env python3
"""
zoomeye_pull.py
~~~~~~~~~~~~~~~
Async wrapper around ZoomEye *host* search.

ZoomEye returns 20 results per page for free accounts.
"""

from __future__ import annotations
import json, os, asyncio, sys, math
from pathlib import Path
from typing import AsyncIterator

import httpx
```

```python
from rich.progress import Progress, BarColumn, TimeRemainingColumn

API_KEY = os.getenv("ZOOMEYE_KEY")
if not API_KEY:
    sys.exit("[-] ZOOMEYE_KEY env var required")

HEAD = {"API-KEY": API_KEY}
BASE = "https://api.zoomeye.org/host/search"

async def page(query: str, p: int) -> list[dict]:
    url = f"{BASE}?query={query}&page={p}"
    async with httpx.AsyncClient(headers=HEAD, timeout=20.0) as cli:
        r = await cli.get(url)
        if r.status_code == 429:       # daily quota exhausted
            raise RuntimeError("ZoomEye quota exceeded")
        r.raise_for_status()
        return r.json()["matches"]

def normalise(hit: dict) -> dict:
    return {
        "ip":        hit["ip"],
        "port":      hit["portinfo"]["port"],
        "transport": hit["portinfo"]["transport"],
        "timestamp": hit["timestamp"],
        "org":       hit.get("org"),
        "banner":    (hit["portinfo"].get("banner") or "")[:8000],
        "source":    "zoomeye",
    }

async def gather(query: str, pages: int, out: Path):
    tmp = out.with_suffix(".tmp")
    pb = Progress("{task.description}", BarColumn(), TimeRemainingColumn())
    task = pb.add_task(f"[cyan]{query}", total=pages)
```

```
    async with tmp.open("w") as fh:
        async with pb:
            for n in range(1, pages + 1):
                try:
                    for match in await page(query, n):
                        fh.write(json.dumps(normalise(match)) + "\n")
                except Exception as e:
                    pb.console.print(f"[red]page {n} failed:[/] {e}")
                pb.advance(task)
                await asyncio.sleep(1.2)  # ZoomEye free tier → 1 req/sec

    tmp.replace(out)
    pb.console.print(f"[green]+[/] ZoomEye data → {out}")

if __name__ == "__main__":
    asyncio.run(
        gather("apache country:\"US\"", pages=25, out=Path("zoomeye.ndjson"))
    )
```

2.5.6 Merging the three providers

```
import pandas as pd, pathlib, json

dfs = [
    pd.read_csv("censys.csv"),
    pd.read_json("shodan.ndjson", lines=True),
    pd.read_json("zoomeye.ndjson", lines=True),
]

df = pd.concat(dfs, ignore_index=True)
df.drop_duplicates(["ip", "port"], inplace=True)
```

```
df.to_csv("internet_scanners.csv", index=False)
print(df.groupby("source").size())
```

2.5.7 Operational / OPSEC Tips

Platform	Credit cost	Stealth note
Shodan	1 credit = 1 banner	The API call doesn't hit the victim, so it's noiseless.
Censys	1 query = 100 hosts	Free tier resets daily at ~00:00 UTC.
ZoomEye	20 hosts per page	Free keys are banned quickly if you burst <1 s delay.

Always throttle to one request per second unless you have a paid key.

2.5.8 Quick Exercises

1. **ASN limiter** – Modify `shodan_pull.py` to stop fetching pages once 5 000 results belong to AS13335 (Cloudflare).
2. **Delta mode** – Add a flag `--since 2024-05-01` and fetch only banners newer than that timestamp (all three APIs support a date filter).
3. **Hybrid scope** – Combine your own masscan CSV with `internet_scanners.csv`, deduplicate, and annotate each host with "scanned_by = {ours, shodan, censys, …}".
4. **Parallel budgets** – Implement a token-bucket that guarantees ≤120 Censys API calls per 24 h, even if multiple team members run the script.
5. **Dashboard** – In a Jupyter notebook, load `internet_scanners.csv`, plot "Top 15 services by banner string" and "Geographic heatmap" (country code -> count).

2.6 - Web Scraping & Anti-Bot Evasion

“If it shows up in a browser we can put it into a DataFrame – as long as we look human enough.”

This module teaches you how to collect OSINT from web sites without tripping rate-limiters, CAPTCHAs, or WAFs.
All examples target Python 3.12, require **no root privileges**, and include extensive inline comments.

2.6.0 Learning objectives

#	Skill
1	Build an **async** scraper that can hit thousands of pages/hour.
2	Rotate TLS fingerprints, User-Agent strings, and timing to bypass naïve defences.
3	Scrape **JS-rendered** content (React, Vue, infinite-scroll).
4	Detect and react to CAPTCHAs / geo blocks automatically.
5	Persist raw HTML + parsed data so downstream modules (e-mail harvest, credential leak search) stay deterministic.

2.6.1 Recommended tool stack

Layer	Library	Reason
Raw HTTP/2	httpx[http2]	HTTP/1.1 + 2, TLS, async, proxies
HTML parser	selectolax	3-5× faster & lower-RAM than BeautifulSoup
Browser (headless)	playwright	Stealth mode, full JS, handles CSP
Fingerprint spoof	tls-client	Chrome 120 JA3 + header mimic out-of-box
Delay/random	asyncio, random, anyio	No extra deps

```
pip install httpx[http2] selectolax playwright tls-client rich anyio fake-useragent
python -m playwright install firefox  # one-time headless browser download
```

2.6.2 Static HTML crawler (heavily commented)

```
"""
static_crawl.py
~~~~~~~~~~~~~~~
Async crawler that:
  • rotates User-Agent per request,
  • randomises Accept-Language / viewport headers,
  • obeys <robots.txt> *path* rules (but ignores crawl-delay),
  • stores both raw HTML and extracted <title>.
"""

from __future__ import annotations
import asyncio, random, re, time, json
from pathlib import Path
from typing import Optional

import httpx
from fake_useragent import UserAgent
from selectolax.parser import HTMLParser
from rich.console import Console
from rich.progress import Progress, BarColumn, TimeElapsedColumn

ua = UserAgent()
console = Console()

def random_headers() -> dict[str, str]:
```

```python
    """Return realistic browser headers."""
    return {
        "User-Agent": ua.random,
        "Accept": "text/html,application/xhtml+xml",
        "Accept-Language": random.choice(
            ["en-US,en;q=0.9", "en-GB,en;q=0.9", "pt-BR,pt;q=0.8"]
        ),
        "Sec-Fetch-Site": "same-origin",
        "Sec-Fetch-Mode": "navigate",
        "Sec-Fetch-Dest": "document",
    }

def parse(html: str) -> str:
    """Extract <title> in a fast, forgiving way."""
    tree = HTMLParser(html)
    title = tree.css_first("title")
    return title.text(strip=True) if title else ""

async def fetch(url: str, client: httpx.AsyncClient) -> Optional[dict]:
    """
    Return dict{"url","title","len"} or None on CAPTCHA / error.
    Detects Cloudflare Turnstile, ReCaptcha, generic 'captcha' keyword.
    """
    try:
        r = await client.get(url, headers=random_headers(), timeout=10.0)
        if r.status_code in (403, 429):
            return None  # blocked
        text = r.text.lower()
        if any(k in text for k in ("recaptcha", "g-recaptcha", "turnstile")):
            return None  # CAPTCHA detected
        return {"url": url, "title": parse(r.text), "len": len(r.text)}
    except Exception:
        return None
```

```python
async def crawl(urls: list[str], out: Path, concurrency: int = 100) -> None:
    out_tmp = out.with_suffix(".tmp")
    async with httpx.AsyncClient(http2=True, follow_redirects=True) as cli, \
            out_tmp.open("w") as fh, \
            Progress("[progress.description]{task.description}",
                     BarColumn(), TimeElapsedColumn()) as bar:

        task = bar.add_task("[cyan]crawl", total=len(urls))
        sem  = asyncio.Semaphore(concurrency)

        async def worker(u: str):
            async with sem:                      # cap parallelism
                meta = await fetch(u, cli)
                if meta:
                    fh.write(json.dumps(meta) + "\n")
                await asyncio.sleep(random.uniform(0.3, 1.3))  # human pause
                bar.advance(task)

        await asyncio.gather(*[worker(u) for u in urls])
    out_tmp.replace(out)
    console.print(f"[green]+[/] Static crawl saved → {out}")

if __name__ == "__main__":
    seed_list = Path("urls.txt").read_text().splitlines()
    asyncio.run(crawl(seed_list, out=Path("static.ndjson")))
```

Inline-comment highlights

- **Random header factory** – `fake_useragent` plus language spoofing kills a lot of basic filters.
- **CAPTCHA heuristics** – bail out early; trying to solve CAPTCHAs at scale is out of scope.
- **Semaphore + sleep** – keeps *effective* RPS ≈ concurrency / (avg sleep + network RTT). Tune as needed.

2.6.3 JavaScript-rendered pages with Playwright

```python
"""
js_render.py
~~~~~~~~~~~~
Loads a page in headless Firefox, waits for network to be idle,
scrolls to bottom (triggering lazy load), returns rendered HTML.
"""

import asyncio
from playwright.async_api import async_playwright

async def rendered_html(url: str, timeout_ms: int = 20_000) -> str:
    async with async_playwright() as pw:
        browser = await pw.firefox.launch(headless=True)
        context = await browser.new_context(
            user_agent="Mozilla/5.0 (Windows NT 10.0; rv:124.0) Gecko/20100101 Firefox/124.0",
            viewport={"width": 1280, "height": 800},
        )
        page = await context.new_page()
        await page.goto(url, timeout=timeout_ms, wait_until="networkidle")
        await page.evaluate("window.scrollTo(0, document.body.scrollHeight)")
        html = await page.content()
        await browser.close()
        return html

if __name__ == "__main__":
    html = asyncio.run(rendered_html("https://example.org/dashboard"))
    print(html[:800])
```

Why Firefox? Playwright's Firefox bundle has fewer detectable *headless* artefacts than Chromium. Switch to Chromium with `stealth-mode` patches if you meet stronger fingerprinting.

2.6.4 Anti-bot evasion cheatsheet (quick reference)

Vector	Simple mitigation
Uniform TLS fingerprint	Use **tls-client** – automatically mimics latest Chrome JA3.
IP rate-limit	1 req/sec baseline, exponential back-off on Retry-After.
TLS session ticket reuse	Keep httpx.AsyncClient alive to reuse tickets & cookies.
Header & payload entropy	Rotate User-Agent, Accept-Language, *and* order of headers.
Mouse/scroll detection	Playwright: call page.mouse.wheel(0, 800) etc.
Geo block	Residential proxies, or public CDN edge (Cloudflare Worker).

2.6.5 Example mini-project – GitHub e-mail leak hunter

```python
"""
gh_leak.py — Search GitHub code for company e-mail addresses via web scraping
(without API token, bypassing the low unauthenticated rate-limit).
"""

import asyncio, re, httpx, urllib.parse, random, json
from selectolax.parser import HTMLParser

COMPANY = "example.com"
DORK    = f'"@{COMPANY}" in:file language:php'

def extract(html: str) -> set[str]:
    tree = HTMLParser(html)
```

```
    code_blobs = tree.css("div.Truncate > a")
    out = set()
    for a in code_blobs:
        blob_url = urllib.parse.urljoin("https://github.com", a.attributes["href"])
        out.add(blob_url)
    return out

async def one_page(page: int) -> set[str]:
    q = urllib.parse.quote_plus(DORK)
    url = f"https://github.com/search?p={page}&q={q}&type=Code"
    async with httpx.AsyncClient(
        headers={"User-Agent": ua.random}, timeout=10.0
    ) as cli:
        html = (await cli.get(url)).text
    return extract(html)

async def main(pages: int = 5):
    blobs = set()
    for p in range(1, pages + 1):
        blobs |= await one_page(p)
        await asyncio.sleep(random.uniform(1.0, 2.0))  # avoid GitHub 429
    Path("gh_blobs.txt").write_text("\n".join(sorted(blobs)))
    print(f"Found {len(blobs)} blob URLs")

if __name__ == "__main__":
    asyncio.run(main())
```

Takeaway: You can perform targeted dorks without hitting the authenticated Search API (which is capped at 30 req/min) – as long as you act like a casual browser with pauses.

2.6.6 Persisting & replaying data

1. **Raw HTML** → `./html_cache/{sha256(url)}.html`
2. **Parsed summary** (title, word counts, emails) → NDJSON or SQLite.
3. **Replay harness** – if you update the parser logic you can re-run against cached HTML without re-scraping (great for offline testing and obeying ToS).

2.6.7 Exercises

#	Challenge	Hint
1	Enhance static_crawl.py to obey crawl-delay from robots.txt per domain.	Use urllib.robotparser.
2	Detect Cloudflare “browser verification” JS loop and retry via Playwright automatically.	Check for id="cf-spinner-..."
3	Extend JS renderer to take full-page screenshots (page.screenshot) and save as PNG.	Path naming with slugified URL.
4	Store all discovered links into a SQLite DB with parent_url → child_url relation.	aiosqlite or sqlite3 in thread
5	Implement token-bucket global rate-limiter shared across multiple crawler processes.	multiprocessing.Manager.

2.6.8 Completion checklist

- Static and JS pages scraped with < 5 % error rate.
- CAPTCHA hits detected and logged (not infinite-looped).
- Headers / TLS fingerprints rotate across at least 5 variants.
- Raw HTML + parsed NDJSON safely written to disk.
- You can replay the parser offline without new HTTP requests.

2.7 - File-Metadata & EXIF Mining

“What you upload tells a story—even if the pixels don’t.”

This module shows how to extract **hidden data** from images, Office documents and PDFs, then turn that data into useable intelligence (GPS heat-maps, author attribution, macro detection). All snippets run on **Python 3.12** and are drenched with inline comments to double as documentation.

2.7.0 Learning Goals

#	Competency
1	Parse EXIF and XMP from JPEG/PNG, convert GPS (DMS → decimal).
2	Reverse-geocode coordinates and build KML/GeoJSON for mapping.
3	Scrape author, company, and creation dates from Office & PDF files.
4	Spot malicious VBA macros and embedded executables.
5	Persist findings in SQLite/CSV for later graph-linking.

2.7.1 Dependencies

```
pip install exifread  pikepdf  oletools  olefile  geopy[timezone]  rich
```

- **exifread** – fast, pure-Python EXIF parser.
- **pikepdf** – PDF metadata & stream extraction (libqpdf wrapper).
- **oletools / olefile** – OLE 2 / VBA macro inspector for .doc, .xls, etc.
- **geopy** – reverse-geocode GPS coordinates; optional timezone look-up.

- **rich** – pretty progress bars and coloured console output.

2.7.2 Directory Layout

```
meta_lab/
 ├─ media/            # photos, screenshots, office docs, PDFs
 ├─ html_cache/       # (from Module 2.6)
 └─ out/
     ├─ exif_geo.json
     ├─ office_meta.csv
     ├─ pdf_meta.csv
     └─ geo.kml
```

2.7.3 GPS Extraction & Reverse-Geocoding (fully commented)

```python
#!/usr/bin/env python3
"""
exif_geo.py
~~~~~~~~~~~
Walks *.jpg / *.jpeg / *.png under ./media, extracts GPS tags, converts
to decimal degrees, reverse-geocodes via Nominatim, then writes:

  • NDJSON  → out/exif_geo.json
  • KML     → out/geo.kml  (for Google Earth / QGIS)
"""

from __future__ import annotations
import exifread, json, pathlib, typing as t
```

```python
from datetime import datetime
from hashlib import sha1

from geopy.geocoders import Nominatim          # OpenStreetMap service
from rich.progress import track

ROOT = pathlib.Path("media")
OUT  = pathlib.Path("out")
OUT.mkdir(exist_ok=True)

# ------------------------------------------------------------------------ #
#  Helpers                                                                 #
# ------------------------------------------------------------------------ #
def dms_to_deg(value, ref: str) -> float:
    """
    Convert EXIF DMS -> decimal degrees.
    value = exif Ratio list   e.g. [52/1, 31/1, 7842/100] (52°31'78.42")
    ref   = 'N', 'S', 'E', or 'W'.
    """
    d, m, s = [x.num / x.den for x in value]     # break Ratio -> float
    deg = d + m / 60 + s / 3600
    return -deg if ref in ("S", "W") else deg

def gps_from_tags(tags: dict) -> tuple[float, float] | None:
    try:
        lat  = dms_to_deg(tags["GPS GPSLatitude"].values,
                          tags["GPS GPSLatitudeRef"].printable)
        lon  = dms_to_deg(tags["GPS GPSLongitude"].values,
                          tags["GPS GPSLongitudeRef"].printable)
        return lat, lon
    except KeyError:
        return None

# ------------------------------------------------------------------------ #
```

```python
#  Main gathering                                                          #
# ------------------------------------------------------------------------- #
geoloc = Nominatim(user_agent="metadata-miner")   # polite user agent
rows: list[dict] = []

for img in track(list(ROOT.rglob("*.[jp][pn]g")), description="EXIF"):
    with img.open("rb") as fh:
        tags = exifread.process_file(fh, details=False)
    gps = gps_from_tags(tags)
    if not gps:
        continue

    # Reverse-geocode (cached in memory by geopy)
    loc = geoloc.reverse(gps, language="en")
    row = {
        "file":       img.as_posix(),
        "sha1":       sha1(img.read_bytes()).hexdigest()[:12],
        "lat":        gps[0],
        "lon":        gps[1],
        "addr":       loc.address if loc else None,
        "timestamp":  tags.get("EXIF DateTimeOriginal", "").printable,
    }
    rows.append(row)

# Write NDJSON
json_out = OUT / "exif_geo.json"
with json_out.open("w") as fh:
    for r in rows:
        fh.write(json.dumps(r) + "\n")
print(f"[+] {len(rows)} geotagged images → {json_out}")

# Simple KML export
kml = ['<?xml version="1.0"?>',
       '<kml xmlns="http://www.opengis.net/kml/2.2"><Document>']
```

```
for r in rows:
    kml.append(f'<Placemark><name>{pathlib.Path(r["file"]).name}</name>'
               f'<description>{r["addr"] or ""}</description>'
               f'<Point><coordinates>{r["lon"]},{r["lat"]},0</coordinates>'
               f'</Point></Placemark>')
kml.append("</Document></kml>")
(OUT / "geo.kml").write_text("\n".join(kml))
print("[+] KML map ready → out/geo.kml")
```

Inline-comment highlights

- `track()` from *rich* gives a free progress bar for potentially thousands of files.
- Uses **SHA-1 prefix** to de-duplicate identical files later.
- Nominatim reverse-geocode is cached by `geopy` per process; if you need heavy volume, self-host an OSM server or use Mapbox.

2.7.4 Office macro + author extraction

```
#!/usr/bin/env python3
"""
office_meta.py
~~~~~~~~~~~~~~
Scans *.doc, *.docm, *.xls, *.ppt, etc. for:
  • core properties  (author, company, created date)
  • presence of VBA macro streams
"""

from oletools.olevba import VBA_Parser, VBA_Scanner
from oletools import olevba
import olefile, csv, pathlib, datetime as dt
```

```python
ROOT = pathlib.Path("media")
CSV  = pathlib.Path("out/office_meta.csv")

fields = ["file", "author", "company", "created", "macros", "suspicious"]
with CSV.open("w", newline="") as fh:
    w = csv.DictWriter(fh, fields)
    w.writeheader()
    for doc in ROOT.rglob("*.do*"):
        # Skip giant files quickly
        if doc.stat().st_size > 50_000_000:
            continue
        try:
            vba = VBA_Parser(str(doc))
            meta = vba.ole_file.get_metadata() if vba.ole_file else None
            suspicious = False
            if vba.detect_vba_macros():
                code = "\n".join([c[3] for c in vba.extract_macros()])
                suspicious = any(
                    s.type == 'Suspicious' for s in VBA_Scanner(code).scan()
                )
            w.writerow({
                "file":      doc.name,
                "author":    meta.author if meta else None,
                "company":   meta.company if meta else None,
                "created":   meta.created.strftime("%Y-%m-%d") if meta else None,
                "macros":    vba.detect_vba_macros(),
                "suspicious": suspicious,
            })
        except olevba.FileOpenError:
            # Not an OLE or OpenXML file
            continue
print(f"[+] Office metadata CSV → {CSV}")
```

Comments show:

- `detect_vba_macros()` vs `VBA_Scanner` difference (presence vs bad-ness).
- Graceful skip for non-OLE formats, giant files or parsing errors.

2.7.5 PDF metadata & embedded file detection

```
#!/usr/bin/env python3
"""
pdf_meta.py
~~~~~~~~~~~
Extracts XMP / InfoDict fields and flags embedded executables.
"""

import pikepdf, pathlib, csv, re
from rich.progress import track

ROOT = pathlib.Path("media")
CSV  = pathlib.Path("out/pdf_meta.csv")

PAT_EXE = re.compile(rb"MZ.{10,100}?This program")   # crude EXE header sig

fields = ["file", "author", "creator", "producer", "created", "embeds"]
with CSV.open("w", newline="") as fh:
    w = csv.DictWriter(fh, fields)
    w.writeheader()

    for pdf in track(list(ROOT.rglob("*.pdf")), description="PDF"):
        try:
            with pikepdf.open(pdf) as doc:
                info = doc.docinfo or {}
```

```
                # Check for embedded files
                embeds = False
                for name, obj in doc.embedded_files.items():
                    if PAT_EXE.search(bytes(obj.read_bytes())):
                        embeds = True
                        break
                w.writerow({
                    "file": pdf.name,
                    "author": info.get("/Author"),
                    "creator": info.get("/Creator"),
                    "producer": info.get("/Producer"),
                    "created": info.get("/CreationDate"),
                    "embeds": embeds,
                })
        except pikepdf.PdfError:
            continue
print(f"[+] PDF metadata CSV → {CSV}")
```

2.7.6 Persisting & joining everything

At this point you have:

- `exif_geo.json` – geo-tagged pictures
- `office_meta.csv` – doc authors & macros
- `pdf_meta.csv` – PDF producers & embedded EXEs

All share a **file SHA-1 prefix** (or full hash) so you can load them into Pandas and correlate with URLs (Module 2.6) or S3 buckets later.

```
import pandas as pd, json, hashlib, pathlib
```

```python
geo  = pd.read_json("out/exif_geo.json", lines=True)
docs = pd.read_csv("out/office_meta.csv")
pdf  = pd.read_csv("out/pdf_meta.csv")

# Join by sha1 if needed
combined = pd.concat([geo, docs, pdf], ignore_index=True)
combined.to_parquet("out/meta_all.parquet")
```

2.7.7 Operational Caveats

- Large reverse-geocode volumes will throttle Nominatim → set `geoloc = Nominatim(…, timeout=3)` and sleep after 1 req/sec.
- Modern iPhones write **HEIC**; exifread doesn't support. Convert with `pyheif` or call `sips` / `magick`.
- Office files may be **OOXML** (`.docx`); oletools handles them transparently.
- PDFs with **owner password** need `pikepdf.open(pdf, password="")` (empty password attempts).

2.7.8 Exercises

#	Task	Hint
1	Write a script that clusters GPS points by addr.postcode and outputs counts per zip.	geopy.location.raw["address"]
2	Detect images whose timestamp differs >12 h from EXIF DateTimeOriginal vs file mtime.	os.path.getmtime
3	Dump all VBA macro code to out/vba/sha1.vba for manual review.	VBA_Parser.extract_macros()
4	Flag PDFs whose /Producer contains "Microsoft Word" but also have embedded files.	Cross-field rule
5	Build an HTML map (Leaflet) that shows photo thumbnails at GPS coordinates on mouse-over.	Use folium or plain Leaflet JS.

2.7.9 Completion Checklist

- EXIF GPS converted & reverse-geocoded.
- KML or GeoJSON ready for analysts / Google Earth.
- Office/PDF authors & suspicious macros catalogued.
- All outputs normalised into a single table for graphing.

2.8 - E-mail, Domain & Breach Correlation

“Map every address, every leak, every alias—from one TXT file to an intel graph.”

This module glues together the recon artefacts produced so far (sub-domains, scraped pages, file metadata) and answers three questions:

1. **Who** are the real people behind the target domain?
2. **Where** have their addresses/passwords appeared in known breaches?
3. **How** can we pivot from one breach or alias to a wider attack surface?

All code below is **Python 3.12**, thoroughly commented, and avoids any paid APIs unless noted.

2.8.0 Road-map

```
[URLs/HTML] ─┐
[PDF/OFFICE] ─┼──► 1. Regex harvest  →  email_raw.txt
[GitHub dorks]┘

email_raw.txt
      │   ── dedup / normalise  →  email_clean.csv
      ▼
┌──────────────────────────────────────────────────────┐
│ 2. Breach look-ups (HIBP, Dehashed/Pastebin search) │
```

```
└──────────────────────────────────────────────────────┘
      │         ── enrich  →  email_breach.csv
      ▼
3. Graph build (NetworkX)  →  breach_graph.gml  → Gephi / Maltego
```

2.8.1 Dependencies

```
pip install email-validator  rich  httpx  networkx  python-louvain
# optional / paid:
# pip install pyhunter  (Hunter.io)
# pip install giturlparse  (for GitHub pivots)
```

2.8.2 Stage 1 – Harvest & Normalise

1-A Extract e-mails from anything (regex + RFC validation)

```
#!/usr/bin/env python3
"""
harvest_emails.py
~~~~~~~~~~~~~~~~~
• Reads raw text blobs (HTML, JSON, office meta).
• Finds candidate e-mails via regex.
• Validates with email-validator (RFC, DNS MX check).
• Writes unique list to email_raw.txt
"""

import re, json, pathlib, sys
```

```python
from email_validator import validate_email, EmailNotValidError
from rich.progress import track

ROOTS = [pathlib.Path("out"), pathlib.Path("html_cache")]  # adjust
RAW_OUT = pathlib.Path("email_raw.txt")

EMAIL_RE = re.compile(r"[\w\.-]+@[\w\.-]+\.\w{2,}", re.I)

seen = set()
with RAW_OUT.open("w") as fh:
    for path in track(
        [p for root in ROOTS for p in root.rglob("*") if p.is_file()],
        description="scan"):
        data = path.read_text(errors="ignore")
        for candidate in EMAIL_RE.findall(data):
            try:
                addr = validate_email(candidate, check_deliverability=False).normalized
                if addr not in seen:
                    seen.add(addr)
                    fh.write(addr + "\n")
            except EmailNotValidError:
                pass
print(f"[+] {len(seen)} unique addresses → {RAW_OUT}")
```

Comments

- **`email-validator`** ensures `john..doe@@` or IDNs are filtered.
- `check_deliverability=False` avoids DNS look-ups (speed); you can enable later.

1-B Add GitHub dork results (Module 2.6)

Just append the scraped blob URLs to the same regex pipeline; no change needed.

2.8.3 Stage 2 – Breach Look-ups

2-A Have I Been Pwned (k-Anon, no API key)

HIBP's "Pwned Password" **range** endpoint lets us test an address hash without sending the full value.

```python
#!/usr/bin/env python3
"""
hibp_anon.py
~~~~~~~~~~~~
• Reads email_raw.txt
• Uses HIBP 'breachedaccount' (paid) if key present,
  otherwise k-Anon *hash prefix* trick (limited: only tells if *password* leaked).
"""

import hashlib, httpx, os, csv, time
from pathlib import Path
from rich.progress import track

EMAILS = Path("email_raw.txt").read_text().splitlines()
APIKEY = os.getenv("HIBP_KEY")          # optional paid key
CSV    = Path("email_breach.csv")

def k_anonymity(email: str) -> bool:
    """True if SHA-1(email) suffix appears in range response."""
    sha1 = hashlib.sha1(email.encode()).hexdigest().upper()
    prefix, tail = sha1[:5], sha1[5:]
    r = httpx.get(f"https://api.pwnedpasswords.com/range/{prefix}", timeout=10.0)
    return any(line.split(":")[0] == tail for line in r.text.splitlines())

def full_api(email: str) -> list[str]:
    """Return list of breach names (paid endpoint)."""
    url = f"https://haveibeenpwned.com/api/v3/breachedaccount/{email}"
```

```
    r = httpx.get(
        url,
        headers={"hibp-api-key": APIKEY, "User-Agent": "intel-course"},
        params={"truncateResponse": "false"},
        timeout=10.0,
    )
    if r.status_code == 404:
        return []
    r.raise_for_status()
    return [b["Name"] for b in r.json()]

rows = []
for em in track(EMAILS, description="HIBP"):
    if APIKEY:
        leaks = full_api(em)
        rows.append({"email": em, "breaches": ";".join(leaks), "count": len(leaks)})
        time.sleep(1.6)                 # paid tiers: 1.5 s rate-limit
    else:
        pwned = k_anonymity(em)
        rows.append({"email": em, "breaches": "pwd-only" if pwned else "", "count": int(pwned)})

with CSV.open("w", newline="") as fh:
    csv.DictWriter(fh, rows[0].keys()).writerows(rows)
print(f"[+] Breach table → {CSV}")
```

Inline comment clarifies the difference between full breach list (paid) and “password appeared somewhere” signal (free).

2-B Pastebin / Dehashed (optional)

- Pastebin scraping is fragile; prefer **Dehashed** paid API (`https://api.dehashed.com/search?query=email:`).
- Normalise responses into the same CSV: `email, breaches, count` .

2.8.4 Stage 3 – Graph Construction (people ⟷ breach ⟷ domain)

```python
#!/usr/bin/env python3
"""
breach_graph.py
~~~~~~~~~~~~~~~
Loads email_breach.csv and outputs NetworkX graph:

  • Node type 'email'
  • Node type 'breach'
  • Edge email → breach (weight=1)
Exports GML for Gephi / Maltego.
"""

import csv, networkx as nx, pathlib

G = nx.Graph()
with pathlib.Path("email_breach.csv").open() as fh:
    rdr = csv.DictReader(fh)
    for row in rdr:
        e = row["email"]
        G.add_node(e, type="email")
        for b in filter(None, row["breaches"].split(";")):
            G.add_node(b, type="breach")
            G.add_edge(e, b)

nx.write_gml(G, "out/breach_graph.gml")
print(f"[+] Graph saved → breach_graph.gml  (nodes={G.number_of_nodes()})")
```

Notes

- **GML** is plain-text; Gephi, Maltego, Cytoscape, Neo4j Desktop all import it.
- You can run `python-louvain` (`community.best_partition(G)`) to highlight clusters.

2.8.5 Bonus — Hunter.io alias expansion (paid/free-tier)

```
from pyhunter import PyHunter
HUNTER = PyHunter(os.getenv("HUNTER_KEY"))
def aliases(domain: str) -> set[str]:
    data = HUNTER.domain_search(domain=domain, limit=50, emails_type="personal")
    return {d["value"] for d in data.get("emails", [])}
```

Merge the alias set back into `email_raw.txt` , dedup, re-run pipeline.

2.8.6 Operational Safety & Ethics

- HIBP ToS **forbid** bulk downloading of breach data (paid key is per-user). Stick to rate-limit, store locally, do not resell.
- Dehashed and similar services *log* every query—assume your searches could be subpoenaed.
- Do **not** use scraped e-mails for phishing unless you have **explicit** written approval (scope of engagement).

2.8.7 Exercises

#	Challenge	Hint
1	Add MX-record validation: drop any e-mail whose domain lacks an MX or AAAA/A record.	dns.asyncresolver

#	Challenge	Hint
2	Score addresses by “breach age” (days since newest breach) and export TOP-100 freshest to CSV.	datetime.fromisoformat
3	For each breach node calculate degree centrality and output the 10 “most common” breaches.	nx.degree_centrality
4	Use GitHub API to pull commit author e-mails for a target organisation and merge into the graph.	GET /repos/{org}/{repo}/commits
5	Build a Streamlit dashboard that lets the team type an e-mail and see all connected breaches graphically.	networkx, pyvis, streamlit

---

2.8.8 Completion Checklist

- Harvested & validated all addresses from previous modules.
- Built `email_breach.csv` with at least breach *presence* info.
- Exported `breach_graph.gml` and inspected clusters in Gephi.
- Understood API rate-limits and ToS for each provider.

2.9 - Ethics, OPSEC & Anti-Correlational Tactics

Recon is only useful if the target never realises you were there and no one gets sued.
This section distils real-world red-team experience into practical rules, scripts and check-lists you can apply **before** running anything from Modules 2.1 – 2.8.

2.9.0 Legal & Ethical Framework

Topic	Minimum due-diligence
Engagement scope	Signed authorisation (Rules of Engagement) listing IP ranges, domains, and explicitly *which third-party services* you may query.
Local law	Check CFAA (US), CMA 1990 (UK), Marco Civil (BR), GDPR (EU). _Passive_queries to Shodan/CT logs are usually legal; active scans often require consent.
Data-handling	Encrypt findings at rest (gpg –symmetric); redact PII before sending to clients.
Disclosure	If you discover non-target data (open Mongo, credentials of another org) you must notify the provider or your customer—never keep it.

2.9.1 OPSEC Layers

1. **Infrastructure separation**
   - Recon VM ➜ results only; no personal e-mail/Slack/SSH keys.
   - Use **cloud throw-away** VPS or a commercial VPN exit, never your home IP.
2. **Transport obfuscation**
Technique	When to use	Python hook
Residential proxy pool	Web scraping heavy JS (Module 2.6)	httpx.AsyncClient(proxies=proxy_url)
Tor / Snowflake	Light DNS brute-force, low bandwidth	socks5h://127.0.0.1:9050
CDN worker egress	Single-domain scraping to blend with legit traffic	https://.workers.dev/fetch?url=
3. **Identity management**
   - Rotate **JA3 / JA3S** fingerprints (`tls-client`).
   - Five distinct User-Agent strings minimum; shuffle header order.
   - Randomise TLS session resumption on bursts (`httpx_limits = httpx.Limits(max_keepalive=5)`).

2.9.2 Traffic-Shaping Snippets

Token-bucket decorator for any async function

```python
import asyncio, time
class Bucket:
    def __init__(self, rate: int, per: int = 60):
        self.rate, self.per = rate, per
        self.allowance = rate
        self.last_check = time.perf_counter()
    async def wait(self):
        while True:
            now = time.perf_counter()
            self.allowance += (now - self.last_check) * (self.rate / self.per)
            self.last_check = now
            if self.allowance >= 1:
                self.allowance -= 1
                return
            await asyncio.sleep(0.2)

bucket = Bucket(rate=60, per=60)       # ≤60 requests / min

async def safe_get(url: str, client):
    await bucket.wait()
    return await client.get(url)
```

Tor identity renewal

```bash
echo -e 'authenticate "your_pass"\nsignal NEWNYM\nquit' | nc -U ~/.tor/control.sock
```

Trigger after every 20 requests in a scraper loop.

2.9.3 Anti-Correlation Checklist

Risk	Mitigation
“Burst → same CIDR → same JA3”	Add 50–200 ms jitter, change proxy/TLS profile every N requests.
DNS wildcard detection on resolver	Query via *public* DoH endpoints (https://dns.google/resolve) not target NS.
Shodan API queries in client logs	Use VPN exit to call Shodan; your IP shows only API traffic, never exploit.
Re-used filenames / metadata leaks	Strip EXIF (exiftool -all= img.jpg) for any screenshots sent in report.
Slack/Teams workplace telemetry	Open reports in *air-gapped* VM; disable DRMs that phone home.

2.9.4 Data-Retention Policy (template)

```
• Raw packets / pcap                : auto-delete after 30 days
• HTML & binary artefacts           : encrypted archive, purge after 90 days
• Final processed CSV / graphs      : retained 1 year (customer audit)
• Keys / secrets (API, VPN, Tor)    : rotated per engagement, nuked on hand-off
```

Implement via a nightly cron:

```
find /recon/raw -type f -mtime +30 -exec shred --remove {} \;
```

2.9.5 Ethics Quick-Reference

Action	Allowed?	Notes
Query Shodan / Censys for target IPs	**Y**	Public data
Download leaked DB from RaidForums	**N**	Illegal possession
Bypass CAPTCHA with paid captcha-farm	**Maybe**	Needs explicit RoE
Mass-mail password reset links	**N**	Social-eng ≠ recon
Report open S3 bucket of third party	**Y**	Coordinated disclosure

---

2.9.6 Exercises

1. **Budget limiter** – Edit any crawler to enforce a daily *global* cap of 1000 requests, persisting the counter to `sqlite` so it survives reboots.
2. **OPSEC audit** – Launch Wireshark on the recon VM and verify that all traffic to `example.com` indeed exits through Tor/IPSEC and never clear.
3. **Ethics scenario** – You find employee payslips on a misconfigured bucket *outside* scope. Write a two-paragraph disclosure memo following ISO 29147.
4. **Correlation test** – Use the target's WAF logs (if you have them) to see whether your jitter + proxy pool still clusters via JA3 or TLS ticket-age. Adjust until indistinguishable.

2.9.7 "Go / No-Go" Gate

Before moving to **Module 3 – Exploitation Scripting** every student must sign off:

- I can explain how my crawler limits RPS and rotates fingerprints.
- I have encrypted all raw data with AES-256 (or stored in a credential-scoped cloud bucket).

- I understand which actions require *separate* customer approval.

1. Passive recon differs from active recon because...

- ◯ A) It never touches the target's network.
- ◯ B) It never uses third-party APIs.
- ◯ C) It only uses ports 80 and 443.
- ◯ D) It requires root privileges.

A resposta correta é:

✅ **A) It never touches the target's network.**

Explicação

Passive reconnaissance (recon passivo) significa coletar informações **sem interagir diretamente com o sistema alvo**. Ou seja, o atacante ou analista obtém dados a partir de **fontes externas**, como:

- DNS records
- WHOIS
- motores de busca
- bases de dados públicas
- OSINT
- certificados TLS públicos
- leaks ou datasets públicos

Isso significa que **nenhum pacote é enviado diretamente para a infraestrutura da vítima**, evitando detecção.

Já **Active reconnaissance** envolve interação direta com o alvo, como:

- **Port scanning (Nmap)**
- **Banner grabbing**
- **Ping sweeps**
- **Service enumeration**

Essas atividades **tocam diretamente a rede do alvo**, podendo ser registradas em logs ou detectadas por IDS/IPS.

Essa distinção é fundamental em **automação de recon e scanning em Python**, onde scripts podem executar scanners ou consultar APIs externas para mapear ativos e serviços expostos.

Por que as outras estão erradas?

B) It never uses third-party APIs.

❌ Errado — recon passivo **frequentemente usa APIs** (Shodan, VirusTotal, IPinfo, etc.).

C) It only uses ports 80 and 443.

❌ Errado — recon passivo não depende de portas específicas.

D) It requires root privileges.

❌ Errado — não tem relação com privilégios.

💡 **Dica de prova:**

Se **não gera tráfego para o alvo** → **Passive Recon**

Se **gera tráfego para o alvo** → **Active Recon**

Exemplo prático no seu dia a dia de **CTI / SOC / OSINT**:

Técnica	Tipo
WHOIS lookup	Passive
Certificate transparency logs	Passive
Shodan search	Passive
Nmap scan	Active
Directory brute force	Active

---

2. When detecting wildcard DNS in a brute-forcer, the recommended technique is to...

- ◯ A) Skip every *.example.com response entirely.
- ◯ B) Resolve a random sub-label and compare its A/AAAA set to later results.
- ◯ C) Disable recursive queries.
- ◯ D) Perform an AXFR first.

A resposta correta é:

✅ **B) Resolve a random sub-label and compare its A/AAAA set to later results.**

Explicação

Quando um domínio possui **wildcard DNS**, qualquer subdomínio inexistente retorna **um IP válido**, por exemplo:

```
*.example.com → 1.2.3.4
```

Isso causa um problema para **DNS brute-forcing**, porque:

```
abc123.example.com → 1.2.3.4
login.example.com → 1.2.3.4
test.example.com → 1.2.3.4
```

Tudo parece existir, mesmo quando não existe.

Técnica recomendada

A técnica padrão usada por ferramentas como:

- **amass**
- **dnsx**
- **massdns**
- **subfinder**

é:

1️⃣ Resolver um **subdomínio aleatório que certamente não existe**, por exemplo:

```
asdhajkshdkajshdk.example.com
```

2️⃣ Registrar os **A/AAAA records retornados**

3️⃣ Comparar os resultados das futuras consultas

Se o IP retornado for **igual ao wildcard**, ele é **descartado**.

Exemplo prático

Wildcard detectado:

```
random123.example.com → 1.2.3.4
```

Resultados:

```
admin.example.com → 1.2.3.4 ❌ (wildcard)
api.example.com → 3.4.5.6 ✅ (válido)
```

Por que as outras estão erradas?

**A) Skip every .example.com response entirely*

❌ Errado — você pode perder subdomínios reais.

C) Disable recursive queries

❌ Não resolve wildcard DNS.

D) Perform an AXFR first

❌ Zone transfer quase sempre está desabilitado.

Regra prática para provas

Wildcard DNS detection = random subdomain baseline comparison

---

3. In module 2.3, ipwhois.lookup_rdap(depth=1) returns which TWO of the following?

- A) TLS JA3 hash
- B) Network CIDR
- C) ASN description
- B and C

A resposta correta é:

✅ **B and C**

- **B) Network CIDR**
- **C) ASN description**

Explicação

A função **ipwhois.lookup_rdap(depth=1)** consulta o **RDAP (Registration Data Access Protocol)** para um endereço IP.

O retorno inclui informações do **bloco de rede e do ASN (Autonomous System)** associado ao IP.

Entre os campos típicos retornados estão:

- **network CIDR** → bloco de rede ao qual o IP pertence Exemplo:

```
8.8.8.0/24
```

-
- **ASN description** → descrição da organização do ASN Exemplo:

```
GOOGLE LLC
```

Isso é muito usado em **Threat Intelligence e OSINT**, por exemplo para:

- identificar o **provedor do IP**
- agrupar infraestrutura adversária
- enriquecer logs de SIEM ou pipelines de detecção

Por que as outras estão erradas?

A) TLS JA3 hash

❌ JA3 é um **fingerprint TLS**, não vem de RDAP.

Exemplo prático em Python

```
from ipwhois import IPWhois

obj = IPWhois("8.8.8.8")
res = obj.lookup_rdap(depth=1)

print(res["network"]["cidr"])
print(res["asn_description"])
```

Saída típica:

```
8.8.8.0/24
GOOGLE LLC
```

💡 **Dica rápida para a prova**

ipwhois.lookup_rdap() → **informação de rede + ASN**

Nunca:

- fingerprints TLS
- hashes
- tráfego de rede

---

4. Which banner-grab method in 2.4 starts with a HEAD / HTTP/1.0 request?

- ◯ A) TLS handshake routine
- ◯ B) Generic tcp_banner() function
- ◯ C) http_head() helper
- ◯ D) ja3_python() library call

A resposta correta é:

✅ **C) http_head() helper**

Explicação

O método **http_head()** realiza banner grabbing em servidores web enviando uma requisição HTTP mínima:

```
HEAD / HTTP/1.0
```

Essa requisição pede **apenas os headers da resposta**, sem o corpo da página, o que é ideal para identificar rapidamente informações como:

- **Server**
- **X-Powered-By**
- **Date**
- **Content-Type**

Exemplo de resposta:

```
HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Tue, 05 Mar 2026 10:22:11 GMT
Content-Type: text/html
```

Isso permite identificar o **software e versão do servidor** de forma rápida e com pouco tráfego.

Por que as outras estão erradas?

A) TLS handshake routine

❌ Usado para negociar criptografia TLS, não envia requisição HTTP.

B) tcp_banner() function

❌ Apenas abre uma conexão TCP e lê o banner (ex: FTP, SMTP), sem enviar HEAD.

D) ja3_python() library call

❌ Usado para **TLS fingerprinting (JA3)**, não para HTTP banner grabbing.

Regra rápida

Método	Técnica
tcp_banner()	banner TCP genérico
http_head()	banner HTTP via HEAD
TLS handshake	fingerprint TLS
JA3	fingerprint cliente TLS

---

5. Shodan's search_cursor() is preferred over search() because it...

- A) Costs zero API credits.
- B) Streams more than 10 000 results page-by-page.
- C) Returns PDFs instead of JSON.
- D) Includes reverse-DNS look-ups.

A resposta correta é:

✅ **B) Streams more than 10 000 results page-by-page.**

Explicação

Na API do **Shodan**, a função:

```
api.search()
```

retorna apenas **uma página de resultados por vez** (tipicamente 100 resultados) e possui limites mais rígidos de paginação.

Já:

```
api.search_cursor()
```

usa um **cursor/iterator** que permite **percorrer grandes volumes de resultados automaticamente**, página por página, sem precisar gerenciar manualmente os parâmetros de paginação.

Ou seja, ele **streama os resultados continuamente**, o que é essencial quando você quer coletar **mais de 10.000 resultados** de uma busca.

Exemplo típico em Python:

```
import shodan

api = shodan.Shodan("API_KEY")

for result in api.search_cursor("apache"):
    print(result["ip_str"], result["port"])
```

O search_cursor():

- faz **auto-pagination**
- retorna resultados **progressivamente**
- facilita **coleta em larga escala**

Isso é muito usado em:

- **Threat Intelligence**
- **asset discovery**
- **infraestrutura adversária mapping**

Por que as outras estão erradas?

A) Costs zero API credits

❌ Ainda consome créditos da API.

C) Returns PDFs instead of JSON

❌ Shodan sempre retorna **JSON**.

D) Includes reverse-DNS look-ups

❌ Não é função do search_cursor().

💡 **Regra rápida para lembrar**

Função	Uso
search()	consultas pequenas
search_cursor()	**coleta massiva de resultados**

---

6. In the static crawler of 2.6, CAPTCHAs are detected by...

- A) Checking for HTTP 200 status.
- B) Scanning HTML for keywords like “recaptcha” or “turnstile”.
- C) Validating HSTS headers.
- D) Comparing page length to robots.txt.

A resposta correta é:

✅ **B) Scanning HTML for keywords like “recaptcha” or “turnstile”.**

Explicação

No **crawler estático**, CAPTCHAs normalmente são detectados analisando o **conteúdo HTML da página** em busca de **strings características** de sistemas de verificação humana.

Exemplos comuns encontrados no HTML:

- recaptcha → **Google reCAPTCHA**
- hcaptcha → **hCaptcha**
- turnstile → **Cloudflare Turnstile**

Um crawler simples pode fazer algo assim em Python:

```
html = response.text.lower()

if "recaptcha" in html or "turnstile" in html:
    print("Possible CAPTCHA detected")
```

Isso funciona porque esses serviços normalmente inserem **scripts ou elementos específicos** no HTML, como:

```
<script src="https://www.google.com/recaptcha/api.js"></script>
<div class="g-recaptcha"></div>
```

Por que as outras estão erradas?

A) Checking for HTTP 200 status

❌ HTTP 200 apenas indica sucesso da requisição.

C) Validating HSTS headers

❌ HSTS está relacionado a **segurança HTTPS**, não CAPTCHA.

D) Comparing page length to robots.txt

❌ Não tem relação com detecção de CAPTCHA.

Regra prática

Em crawlers simples:

Técnica	O que detectar
HTML keyword scan	CAPTCHA
Header analysis	segurança HTTP
status codes	disponibilidade
DOM parsing	links e conteúdo

---

7. Converting EXIF GPS degrees-minutes-seconds (DMS) to decimal degrees requires which formula?

- ◯ A) deg + min/60 + sec/3600
- ◯ B) deg × min × sec
- ◯ C) (deg – min)/sec
- ◯ D) deg + min × 60 + sec × 3600

A resposta correta é:

✅ **A) deg + min/60 + sec/3600**

Explicação

Para converter coordenadas **DMS (Degrees-Minutes-Seconds)** para **graus decimais**, usamos a fórmula:

$$\text{Decimal Degrees} = deg + \frac{min}{60} + \frac{sec}{3600}$$

Porque:

- **1 minuto = 1/60 de grau**
- **1 segundo = 1/3600 de grau**

Exemplo

Coordenada EXIF:

```
40° 26' 46"
```

Conversão:

```
40 + 26/60 + 46/3600
```

Resultado:

```
40.446111°
```

Observação importante (EXIF GPS)

Ao lidar com **metadata EXIF**, também é necessário considerar a direção:

Direção	Sinal
N	positivo
S	negativo
E	positivo
W	negativo

Exemplo em Python:

```
decimal = deg + minutes/60 + seconds/3600

if ref in ["S", "W"]:
    decimal = -decimal
```

Isso é muito usado em **OSINT, geolocalização e análise forense de imagens**, por exemplo quando se extrai GPS de fotos usando bibliotecas como:

- exifread
- Pillow

- piexif

✅ **Resposta final: A**

8. In 2.8, the “k-Anon” HIBP technique avoids sending the full e-mail by...

- ◯ A) Hashing with MD5.
- ◯ B) Simply URL-encoding the address.
- ◯ C) Encrypting the address with RSA.
- ◯ D) Sending the first 5 characters of a SHA-1 hash and matching the suffix locally.

A resposta correta é:

✅ **D) Sending the first 5 characters of a SHA-1 hash and matching the suffix locally.**

Explicação

A técnica **k-Anonymity** usada pela API do **Have I Been Pwned (HIBP)** permite verificar vazamentos **sem enviar o e-mail ou senha completa** para o servidor.

O processo funciona assim:

1️⃣ O cliente calcula o **hash SHA-1** da senha ou e-mail.

Exemplo:

```
password → 5BAA6...
```

2️⃣ Apenas os **primeiros 5 caracteres do hash** são enviados para a API.

```
5BAA6
```

3️⃣ O servidor retorna **todos os hashes que começam com esse prefixo**.

4️⃣ O cliente **compara localmente o restante do hash (suffix)** para verificar se há correspondência.

Ou seja:

```
prefix → enviado ao servidor
suffix → comparado localmente
```

Isso garante que o servidor **nunca veja o valor completo original**.

Exemplo simplificado em Python

```
import hashlib
import requests

password = "password"
sha1 = hashlib.sha1(password.encode()).hexdigest().upper()

prefix = sha1[:5]
suffix = sha1[5:]

resp = requests.get(f"https://api.pwnedpasswords.com/range/{prefix}")

for line in resp.text.splitlines():
    hash_suffix, count = line.split(":")
```

```
    if hash_suffix == suffix:
        print("Password found in breaches:", count)
```

Por que as outras estão erradas?

A) Hashing with MD5

❌ HIBP usa **SHA-1**, não MD5.

B) URL-encoding

❌ Não protege a privacidade.

C) Encrypting with RSA

❌ A técnica não usa criptografia assimétrica.

💡 Resumo

Técnica	O que envia
k-Anonymity (HIBP)	**prefixo SHA-1 (5 chars)**
comparação	**suffix local**

---

9. According to the OPSEC checklist (2.9), a JA3 fingerprint helps evade correlation when you...

- ◯ A) Rotate it alongside User-Agent strings.
- ◯ B) Keep it constant across requests.
- ◯ C) Disable TLS entirely.
- ◯ D) Use only HTTP/1.0.

A resposta correta é:

✅ **A) Rotate it alongside User-Agent strings.**

Explicação

O **JA3 fingerprint** é uma técnica usada para identificar clientes TLS analisando os parâmetros do **TLS ClientHello**, como:

- versões TLS suportadas
- cipher suites
- extensões TLS
- elliptic curves

Esses parâmetros são combinados e geram um **hash JA3**, que funciona como uma **impressão digital do cliente TLS**.

Ferramentas de segurança (IDS, SIEM, EDR) usam JA3 para **correlacionar tráfego e identificar ferramentas automatizadas**.

OPSEC / evasão

Se você usar sempre o mesmo JA3 fingerprint:

```
Scanner → JA3 X
Scanner → JA3 X
```

```
Scanner → JA3 X
```

os sistemas defensivos conseguem **correlacionar facilmente as requisições**.

Por isso, a recomendação de **OPSEC** é:

➡️ **rotacionar o JA3 juntamente com o User-Agent**

Exemplo:

Request	User-Agent	JA3
1	Chrome	JA3-Chrome
2	Firefox	JA3-Firefox
3	Safari	JA3-Safari

Isso faz o tráfego parecer **originado de diferentes clientes legítimos**.

Por que as outras estão erradas?

B) Keep it constant across requests

❌ Facilita a correlação.

C) Disable TLS entirely

❌ Irrealista e suspeito.

D) Use only HTTP/1.0

❌ Não tem relação com JA3.

Regra rápida

Técnica	Objetivo
Rotacionar User-Agent	evitar fingerprint HTTP
Rotacionar JA3	evitar fingerprint TLS

💡 Curiosidade interessante para **Threat Intelligence / DFIR**:

Ferramentas como **Zeek, Suricata e Splunk** frequentemente usam JA3 para detectar malware, por exemplo:

- **Cobalt Strike**
- **TrickBot**
- **Emotet**

porque muitas famílias de malware reutilizam o **mesmo fingerprint TLS**.

10. Recommended retention for raw recon data (template in 2.9) is...

- ◯ A) Keep everything forever.
- ◯ B) Delete after 7 days.
- ◯ C) Encrypt and purge PCAPs after 30 days; purge HTML after 90 days.
- ◯ D) Only keep what fits in 1 GB.

A resposta correta é:

✅ **C) Encrypt and purge PCAPs after 30 days; purge HTML after 90 days.**

Explicação

No **checklist de OPSEC (seção 2.9)**, a recomendação é manter **retenção limitada e controlada** para dados de recon, porque eles podem conter:

- **tráfego capturado (PCAPs)**
- **conteúdo de páginas HTML**
- **metadados de infraestrutura**
- **artefatos potencialmente sensíveis**

A política sugerida é:

Tipo de dado	Retenção
PCAPs	criptografar e apagar após **30 dias**
HTML / conteúdo coletado	apagar após **90 dias**

Isso segue boas práticas de:

- **minimização de dados**
- **redução de risco operacional**
- **OPSEC**
- **compliance de privacidade**

Por que as outras estão erradas?

A) Keep everything forever

❌ Risco de exposição e armazenamento desnecessário.

B) Delete after 7 days

❌ Curto demais para análises e correlações.

D) Only keep what fits in 1GB

❌ Não é uma política de retenção adequada.

✅ **Resposta final: C**